

# hive

## A User's Manual

*A fleet console for coding agents.*

Your laptop sleeps. The agents keep working.

Last revised: September 16, 2026

torsor lab

[torsor.org](https://torsor.org)

# Contents

---

|                                                  |           |
|--------------------------------------------------|-----------|
| <b>Preface: The Laptop Is the Wrong Place</b>    | <b>iv</b> |
| <b>1 The Fleet</b>                               | <b>1</b>  |
| Four roles . . . . .                             | 1         |
| What talks to what . . . . .                     | 2         |
| What each machine has to provide . . . . .       | 3         |
| The tailnet is the authentication . . . . .      | 5         |
| What a session actually is . . . . .             | 5         |
| What happens when a piece goes missing . . . . . | 6         |
| <b>2 Quick Start: Let an Agent Do It</b>         | <b>8</b>  |
| Before you start . . . . .                       | 8         |
| What to ask for . . . . .                        | 9         |
| What it can do for you . . . . .                 | 10        |
| What it will have to ask you for . . . . .       | 11        |
| How to tell it actually worked . . . . .         | 11        |
| Then stop and read about the consoles . . . . .  | 13        |
| <b>3 Standing Up the Fleet</b>                   | <b>14</b> |
| What a converge does . . . . .                   | 14        |
| One: Ansible on your laptop . . . . .            | 15        |
| Two: describe your fleet . . . . .               | 15        |

|                                                  |           |
|--------------------------------------------------|-----------|
| Three: the sudo detour, if you need it . . . . . | 17        |
| Four: converge . . . . .                         | 17        |
| Five: linger . . . . .                           | 18        |
| Six: verify . . . . .                            | 18        |
| Seven: the phone console . . . . .               | 19        |
| Where everything landed . . . . .                | 19        |
| Adding a box later . . . . .                     | 20        |
| Pinning a version . . . . .                      | 20        |
| <b>4 Three Consoles</b>                          | <b>21</b> |
| The terminal . . . . .                           | 21        |
| The Mac panel . . . . .                          | 22        |
| The phone . . . . .                              | 23        |
| Attach: the escape hatch . . . . .               | 24        |
| Which one to reach for . . . . .                 | 25        |
| <b>5 Running Agents</b>                          | <b>26</b> |
| The four decisions . . . . .                     | 26        |
| Passing arguments to the agent . . . . .         | 27        |
| The permission trap . . . . .                    | 27        |
| Stop, restart, kill, resume . . . . .            | 28        |
| What happens on the box . . . . .                | 29        |
| Stars . . . . .                                  | 29        |
| Looking around a box before you start . . . . .  | 30        |
| <b>6 Chat and Transcripts</b>                    | <b>31</b> |
| Saying something . . . . .                       | 31        |
| Reading the conversation . . . . .               | 32        |
| When the conversation goes stale . . . . .       | 32        |
| When there's nothing to read yet . . . . .       | 34        |
| <b>7 Keeping It Running</b>                      | <b>35</b> |
| Pushing new code . . . . .                       | 35        |

|                                   |           |
|-----------------------------------|-----------|
| After a reboot . . . . .          | 36        |
| The failures you'll see . . . . . | 36        |
| Health at a glance . . . . .      | 38        |
| Housekeeping . . . . .            | 38        |
| Moving the hub . . . . .          | 38        |
| What isn't automatic . . . . .    | 38        |
| <b>A Quick Reference</b>          | <b>40</b> |
| Fleet and sessions . . . . .      | 40        |
| Daemons . . . . .                 | 41        |
| Ports . . . . .                   | 42        |
| Configuration files . . . . .     | 42        |
| Environment variables . . . . .   | 44        |
| HTTP API . . . . .                | 44        |
| Provisioning . . . . .            | 45        |
| Building from source . . . . .    | 46        |
| Running it locally . . . . .      | 47        |
| When something is wrong . . . . . | 48        |

# Preface: The Laptop Is the Wrong Place

---

The agent is still running and you need to close the lid.

Maybe you're leaving the office. Maybe the battery is at 4%. Maybe you just want to go to dinner. The agent doesn't care — it's forty minutes into a refactor, and the moment your laptop sleeps, so does the work. You've probably already tried the obvious fix: run it on a machine that stays on, reach it over SSH, keep it alive inside `tmux`. That works. It works right up until there are four such machines, six such sessions, and no memory of which box you left the interesting one on.

At that point you're not running agents. You're doing archaeology. You SSH somewhere, run `tmux ls`, squint at names you chose in a hurry, attach to the wrong one, detach, try the next box. And because that ritual is tedious, you stop starting long jobs — which is the real cost, and the one that's hard to notice.

**hive** is a console for agent sessions that live on other machines.

You keep the agents where they belong — on boxes that stay powered on — and **hive** gives you one view of all of them: what's running where, what each one last said, and a way to answer it. From a terminal.

From a native app on your Mac. From your phone while you're standing in line for coffee. Same fleet, same sessions, three different places to reach them.

The part that takes a moment to accept is that *hive* is not a single program you install. It is several programs and a handful of configuration files, spread across the machines you already have, and the arrangement is the product. One program runs on each box and owns the truth about what's running there. One program runs on a single always-on machine and collects that truth from all the boxes. The things you actually touch — the terminal command, the Mac panel, the phone console — hold no state at all; they ask the collector. The configuration files say which machine is which.

That shape is why your laptop is allowed to sleep, and it's why *hive* asks more of you up front than a one-line install would. You need machines that stay on, and you need them able to find each other. *The Fleet* lays out exactly what those machines are and what each one has to provide. Read it before you install anything; it's short, and it's the chapter that makes the rest make sense.

*Quick Start* is the fast path, and an unusual one: hand the repository to a coding agent and let it do the setup. This works better than it sounds, because *hive*'s own conventions are written down where an agent will find them — but only up to a point, and the chapter is mostly about where that point is. What an agent can do unattended, what it will have to ask you for, and how you tell a real success from a plausible-looking failure. *Standing Up the Fleet* covers the same ground by hand, slowly, for when you'd rather drive.

Then the payoff. *Three Consoles* is the `hive` command, the Mac panel, the phone console — what each is good at, and why chat from a phone turns out to be the feature that changes how you work. *Running Agents* is starting agents and managing their lifecycles. *Chat and*

*Transcripts* is talking to a running agent and reading what it said, including the one genuinely confusing thing in the system, which is how a session gets attached to a transcript and what to do when that attachment goes stale. *Keeping It Running* is keeping a fleet healthy over months.

The appendix is the exhaustive part — every command, flag, endpoint, config field, and path. Nothing in the chapters tries to be complete. The appendix does.

# The Fleet

---

Most software asks what operating system you have. `hive` asks something harder: which of your machines never turns off.

That's the requirement underneath everything else in this manual. `hive` exists to keep agent sessions alive while you walk away, and nothing in the software can supply a computer that stays powered on. So before any install command, it's worth being concrete about which machines you're going to use and what job each one does. There are four jobs. One machine can hold more than one of them.

## Four roles

An **agent box** is a machine that stays on and actually runs the agents. Each one runs a small daemon called `hive-host`, which starts agents inside `tmux`, keeps a file describing each session, and answers questions about them over HTTP. The box is the truth: if `hive-host` says a session is running, it's because the `tmux` session is really there, on that machine, right now. Most fleets have two to four boxes. One is enough.

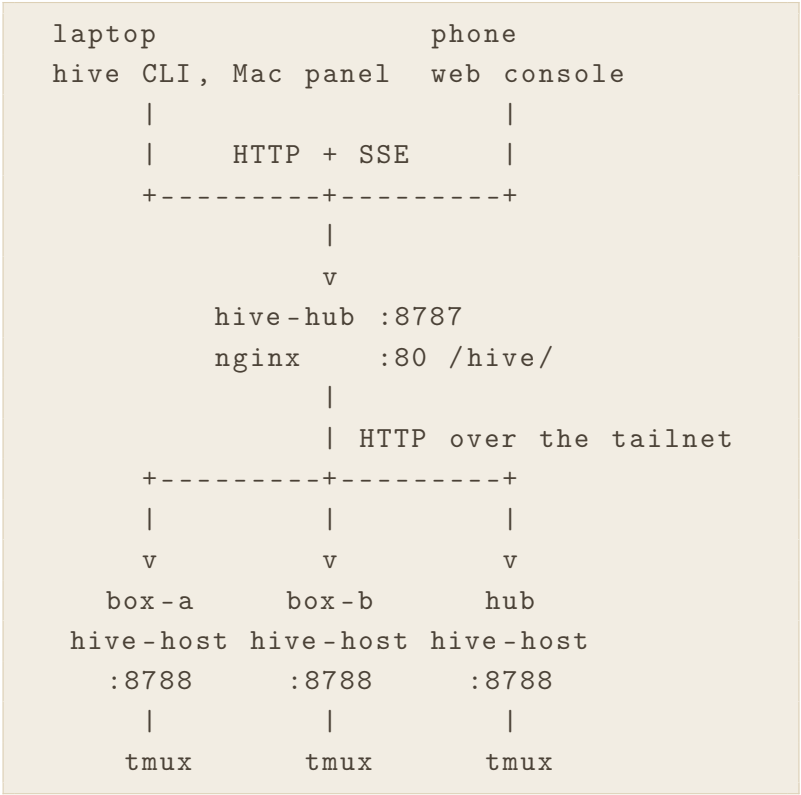
The **hub** is one designated always-on machine that collects. Every few seconds it asks each box what it's running and caches the answer. It also forwards commands — your “stop this session” goes to the hub, and the hub passes it to the right box. The hub is an ordinary agent box that has been given this extra job, so it can run agents too. What

makes it the hub is that everything else depends on it being reachable, which means you should pick the machine you trust most to stay up.

A **desktop client** is your laptop — the `hive` command in a terminal, or the Mac panel, or both. Clients keep no state. They ask the hub and render what comes back, which is precisely why the laptop is allowed to sleep, get closed, or fly across an ocean.

A **phone** is the fourth. It loads a small web console served from the hub and talks to the same API your laptop uses. You install nothing on the phone beyond Tailscale itself.

What talks to what



Two things in that picture are worth saying out loud.

First, the clients never connect to a box. They talk to the hub, and only to the hub. This is what makes the phone console possible at all — one address to reach, one thing to be reachable.

Second, there's no SSH on that diagram. Status, chat, starting an agent, stopping one: all HTTP. SSH appears in exactly two places in this manual — installing the software in the first place, and attaching a real terminal to a running agent when you want to type at it directly.

The hub is listed twice above, as `hub:8787` and `hub:8788`. That isn't a typo. A hub that also runs agents is running two daemons: the collector on 8787 and its own `hive-host` on 8788. The collector polls its own box over HTTP just like any other.

## What each machine has to provide

Every agent box, including the hub

A Linux machine with `apt` — the provisioning assumes the Debian/Ubuntu family. It has to stay powered on and awake; a box that suspends is a box whose agents stop.

You need four things on it before `hive` can be installed, and the installer cannot supply any of them for you:

- **Tailscale, up and authenticated.** Every `hive` daemon binds to the machine's Tailscale address and refuses to bind anywhere else.
- **SSH access** from your laptop, under the account that will own the agents.
- `sudo` for that account, for the first install only.
- **The agent itself** — `claude`, or `codex`, or both — installed *and already signed in* for that same account.

That last one is the requirement people miss. `hive` does not install coding agents and does not hold credentials for them. It runs whatever `claude` or `codex` it finds on the box, as you, using the login you already did there. A box with no agent installed will provision perfectly and then fail the moment you ask it to start a session.

Everything else — `git`, `tmux`, a compiler, a Rust toolchain, the `hive` binaries themselves — is installed for you during setup.

`hive` builds from source *on each box*. The first install on a new machine spends several minutes compiling Rust, which is normal and which you should not interrupt. Copying a binary from your Mac does not work; it's the wrong architecture and the wrong operating system.

The hub, additionally

Whatever a box needs, plus a web server for the phone console, which the installer configures. Nothing more is asked of the machine. What's asked of you is the choice: this is the machine that everything else routes through, and if it's off, every client goes blind at once. The agents keep running. You just can't see them.

Your laptop

For the terminal, one program: the `hive` command, plus a two-line config file naming your hub.

For setting up the fleet, additionally `ansible-core` and SSH keys to the boxes. Only the machine that provisions needs this — a second laptop can use `hive` perfectly well with nothing but the `hive` command and the hub's address.

For the Mac panel, a macOS machine and a Node toolchain to build it. The panel is a native application that speaks to the hub over HTTP;

it is not a browser window pointed at the hub.

Your phone

Tailscale, signed in to the same network. That's the whole list. The console is a web page served by the hub, so there's nothing to install and nothing to keep updated.

## The tailnet is the authentication

hive has no accounts, no passwords, no tokens, and no TLS. Being on your Tailscale network *is* the credential.

This is a deliberate trade, and it's the right one for a personal fleet — it's what makes a phone console a fifteen-minute feature instead of an authentication project. But it has a consequence you should hold clearly in mind.

Anyone who can reach your tailnet can read your agents' conversations, send them messages, start new agents on your machines, and stop running ones. There is no second check. Treat tailnet membership exactly as seriously as you would treat an SSH key to the same machines, and don't add devices you don't control.

The software enforces its half of the bargain strictly. Both daemons refuse to start unless their bind address is a Tailscale address — they will not bind to 0.0.0.0, and they will not bind to loopback unless you explicitly pass a development flag. You cannot accidentally publish your fleet to the open internet, because the program will stop rather than let you.

## What a session actually is

One agent, running on one box, under a name you chose.

The name is the handle for everything — a box and a task name identify a session completely:

```
hive stop box-a period-index
```

On the box, that name becomes two things: a `tmux` session named `hive-` followed by the task, and a small metadata file recording the working directory, which agent it is, and when it started. Nothing else about a session lives anywhere — there's no central database, and the hub's copy is just a cache it refreshes every few seconds.

The practical consequence: the box is always right, and everything else is a view. If the hub says something the box disagrees with, the box wins, and the disagreement resolves itself within seconds.

## What happens when a piece goes missing

**Your laptop sleeps.** Nothing happens. This is the whole point. Clients hold no state and nothing depends on them; agents keep working and you pick up where you left off.

**A box goes down.** Its row in the fleet table reads (`unreachable`) with the error the hub got. Every other box is unaffected — one dead machine doesn't take down your view of the rest.

**The hub goes down.** Every client goes blind at once: no status, no chat, no starting or stopping anything. The agents themselves are entirely unaffected, still running in `tmux` on their boxes. When the hub comes back, so does your view, with no reconciliation needed. If you need to reach an agent while the hub is down, SSH to its box and attach to `tmux` directly — that path never stops working.

hive is a console, not a supervisor. It shows you what your agents are doing and lets you talk to them. It does not restart an agent that crashed, resume one after a reboot, or notice that a session has been stuck waiting for input since Tuesday. Those remain your job.

That's the whole architecture. Two daemons, some configuration files saying which machine is which, and three consoles that hold nothing. The next chapter puts it on real machines.

# Quick Start: Let an Agent Do It

---

There's a pleasing circularity to setting up hive by handing the repository to a coding agent and asking it to install the thing that will manage coding agents. It also happens to be the fastest route, because hive's conventions are written down inside the repository in the places an agent looks first.

It is not, however, a route you can start and walk away from. This chapter is mostly about where the agent's usefulness stops — what it can do unattended, what it has to ask you for, and how to tell a real success from a confident-sounding failure.

## Before you start

Have this ready. None of it can be produced by the agent, and each missing piece stops the install at a different point.

- The names of your boxes, as you SSH to them, and which one is the hub.
- Tailscale running and signed in on every box, on your laptop, and on your phone.
- Working SSH from your laptop to each box, and the `sudo` password for the account you SSH as.
- `claude` or `codex` installed and signed in on each box, under that same account.

- A URL the boxes can clone hive from. Each box fetches the source itself and builds it, so a repository that only exists on your laptop is not enough.

The last two are the ones that quietly derail an otherwise clean install. Both boil down to the same question — can *the box*, acting as you, get what it needs without you sitting there? An agent signed in only on your laptop and a repository reachable only from your laptop will both pass every check until the moment they don't.

## What to ask for

Open an agent session in your clone of the hive repository and describe where you're starting from. Concretely, and with your real machine names:

```
Read README.md, CLAUDE.md, and
  ansible/README.md.

I want to set up a hive fleet. My boxes
  are box-a and box-b,
and hub is the always-on machine that
  should run hive-hub.
All three are on my tailnet and I can SSH
  to them.

Walk me through it. Stop and ask before
  anything that needs
my password or changes a machine.
```

The last line matters more than the rest. Provisioning is the kind of task an agent will happily drive to completion, and you want the first

converge to be something you watched.

## What it can do for you

A capable agent working in that repository will get the shape right, because the repository tells it the shape. Expect it to handle:

**The configuration files, correctly placed.** hive keeps everything about *your* fleet outside the repository, under `~/.hive/fleet/`. There are two files, both copied from examples in the repo and then edited: an inventory naming your boxes and marking which one is the hub, and a vars file saying where the boxes clone from. Getting these right by hand means reading a README; an agent just does it.

**Explaining what the converge will do** before you run it, in the order it will happen: install packages, install Rust, clone hive on each box, build it there, install the daemons as services, then generate your laptop's client files.

**Reading the failures.** This is where an agent genuinely earns its place. Ansible failures are long and the real error is rarely in the last line. An agent reads the whole thing and tells you which host failed at which task.

**The known traps.** Two are documented in the repository, and an agent that read the docs will raise them before they bite. On recent Ubuntu, `sudo` may be routed through an implementation that Ansible's password prompt cannot drive, and the fix is a one-line switch run once per box *before* the first converge. Separately, the daemons run as user services, which by default stop when you log out — a setting called *linger* has to be enabled once per box or your fleet goes quiet whenever your SSH session ends.

## What it will have to ask you for

**Your sudo password.** The first converge installs system packages and creates a directory owned by root. Ansible prompts for this, interactively, and an agent cannot type it for you — nor should you want it to.

**Which machine is the hub.** There's no way to infer it. It's a judgment about which of your machines is most reliably on.

**Authenticating Tailscale and the coding agents.** Both are browser-based logins on each box. The install will complete without them and then fail in ways that look unrelated: a box that can't be reached, or a session that starts and immediately dies.

**Permission to push.** The boxes clone from a URL, so whatever you want deployed has to be pushed there first. If the agent has been editing hive itself, the code on the boxes is whatever was pushed — not what's in your working directory.

## How to tell it actually worked

An install can report success at every step and still leave you with a fleet that doesn't work, so check it yourself. Four checks, in order, each one testing a different link in the chain.

**One.** On each box, is the daemon running?

```
ssh box-a 'systemctl --user is-active  
hive-host'
```

You want *active*. Anything else and that box isn't in the fleet yet.

**Two.** Is it actually answering on the network, rather than merely running?

```
ssh box-a 'curl -s http://$(tailscale ip
-4):8788/v1/sessions '
```

You want `[]` — an empty list of sessions. An empty response instead means the daemon is up but not listening where you think it is.

**Three.** Can the hub see everyone? From your laptop:

```
hive status
```

Every box should be listed, each saying (no `sessions`). A box showing (`unreachable`) is the interesting case, and it means the hub couldn't reach it — which is a different problem from that box being down, and *Keeping It Running* separates them.

**Four.** Start something real and watch it appear:

```
hive run box-a hello
/home/you/some/project
hive status
```

The session should show as `running`, with the last line of the agent's output beside it. This is the check that exercises everything at once — the client, the hub, the box, `tmux`, and the coding agent's own login. If the first three passed and this one fails, the problem is almost always that last item.

Don't accept "the playbook completed successfully" as evidence. Ansible reports on the tasks it ran, not on whether you have a working fleet — and the two most common failures, an unauthenticated Tailscale and an unauthenticated coding agent, both leave a converge looking perfectly clean. Run the four checks.

## Then stop and read about the consoles

Once `hive status` lists your boxes, you're done with setup and you haven't yet used `hive` for anything. The terminal is the least interesting of its three faces; *Three Consoles* covers the Mac panel and the phone console, which are what make a fleet feel like something you can keep an eye on rather than something you have to remember to check.

If the install didn't go cleanly, or you'd rather understand the machinery before trusting it, *Standing Up the Fleet* walks the same path by hand.

# Standing Up the Fleet

---

The same install as *Quick Start*, done deliberately. Read this if the agent route stalled, if you'd rather understand what's happening to your machines, or if you're adding a box to a fleet that already exists.

Provisioning uses Ansible, and it runs from your laptop. Your laptop is the only machine that needs it — the boxes need nothing but SSH and a willingness to compile.

## What a converge does

In order, on every box:

1. Install system packages: `git`, `tmux`, a compiler, the libraries `hive` needs.
2. Install a Rust toolchain, if the box doesn't have one.
3. Clone `hive` at a pinned version into `/srv/lab/hive`.
4. Build it there, from source.
5. Install `hive-host` as a service and start it.
6. On the hub only: the same for `hive-hub`, plus the phone console.
7. Check that `tmux` and a coding agent are present, and say so if they aren't.

Then, on your laptop, it writes the two client files that tell `hive` where your fleet is.

Two things to notice. The build happens on each box — the first converge on a new machine spends several minutes in *cargo*, which is expected. And the last step only *checks* for a coding agent; installing and signing in to *claude* or *codex* on each box is yours to do, and the converge will finish happily without it.

## One: Ansible on your laptop

```
uv tool install ansible-core
```

## Two: describe your fleet

Nothing about your machines lives in the hive repository — it's public, and it stays that way. Your fleet is described by two files under `~/.hive/fleet/`, copied from examples in the repo and then edited. From your clone:

```
mkdir -p ~/.hive/fleet/group_vars/all
cd ansible
cp inventory.example.yml
  ~/.hive/fleet/inventory.yml
cp group_vars.example/all/vars.yml
  ~/.hive/fleet/group_vars/all/vars.yml
```

The inventory names your machines and marks one as the hub:

```
hive_hosts:
  hosts:
    box-a:
    box-b:
  children:
    hive_hub:

hive_hub:
  hosts:
```

```
hub:
vars:
    hive_hub_enabled: true

control:
    hosts:
        localhost:
            ansible_connection: local
```

Use the names you actually SSH to. The hub sits in its own group that is a child of `hive_hosts`, which is what makes it both the collector and an ordinary agent box.

The vars file says where the boxes get their code:

```
hive_repo_url:
    https://github.com/OWNER/hive.git
hive_ref: main
hive_checkout_dir: /srv/lab/hive
hive_hub_host: hub
```

`hive_repo_url` is fetched *by the boxes*, not by your laptop. A path that only resolves locally, or a private repository the boxes can't authenticate to, fails at the clone step. Whatever you want running on the fleet has to be pushed to that URL first.

Every command below runs from the `ansible` directory of your clone. That is where the configuration lives that points Ansible at your `~/.hive/fleet/` files.

### Three: the sudo detour, if you need it

Recent Ubuntu releases may route `sudo` through a reimplementation that Ansible's password prompt cannot drive. The symptom is a converge that hangs at privilege escalation and eventually times out. Classic `sudo` is already installed; switch to it once per affected box, before the first converge:

```
ssh box-a 'sudo update-alternatives --set  
sudo /usr/bin/sudo.ws'
```

### Four: converge

Start with one box you don't mind breaking, plus your laptop:

```
ansible-playbook site.yml -K --limit  
box-a,localhost
```

`-K` prompts for your `sudo` password, which the first run needs to install packages and create the checkout directory. Later runs usually don't.

Including `localhost` is not decoration. The laptop's client files are written by a separate play that only runs when `localhost` is in scope. Limit to a box alone and the boxes get provisioned while your `hive` command still doesn't know they exist.

When that box looks right, do the rest:

```
ansible-playbook site.yml -K
```

To see what a run would change without changing it:

```
ansible-playbook site.yml -K --check
--diff
```

## Five: linger

The daemons run as *user* services, which by default stop when you log out. Without a setting called *linger*, your fleet goes quiet every time an SSH session ends — and the symptom is maddening, because boxes appear and disappear depending on whether you happen to be logged in.

The converge enables it, but that step needs a password, so it's easy to skip on a run without *-K*. To be sure:

```
ssh box-a 'sudo loginctl enable-linger
$USER'
```

## Six: verify

The four checks from *Quick Start*, in order: the service is active on each box; it answers on its Tailscale address; *hive status* lists everything from your laptop; and a real session starts and appears.

```
ssh box-a 'systemctl --user is-active
hive-host'
ssh box-a 'curl -s http://$(tailscale ip
-4):8788/v1/sessions'
hive status
```

On the hub, also confirm the collector is up and knows about everyone:

```
ssh hub 'systemctl --user is-active
hive-hub'
ssh hub 'cat ~/.hive/hosts'
```

## Seven: the phone console

Served by the hub, and deployed separately because it's built on your laptop and copied over:

```
ansible-playbook site.yml -K --tags web
--limit hive_hub
```

Then open `http://hub/hive/` on a phone that's on your tailnet.

## Where everything landed

| Path                | Machine                         |
|---------------------|---------------------------------|
| /srv/lab/hive       | each box: source checkout       |
| /srv/lab/hive/bin   | each box: the built daemons     |
| ~/.hive/sessions    | each box: one file per session  |
| ~/.hive/host.toml   | each box: bind address and port |
| ~/.hive/hosts       | hub: generated fleet list       |
| ~/.hive/hub.toml    | hub: collector settings         |
| ~/.hive/client.toml | laptop: which hub to talk to    |
| ~/.hive/fleet/      | laptop: your inventory and vars |

Two of these are generated and should not be hand-edited: the hub's fleet list and the laptop's copy of it, both written from your inventory on every converge. To add or remove a machine, edit the inventory and converge — editing the generated file works until the next run silently undoes it.

`client.toml` is the exception. It's seeded once, when missing, and then it's yours; edit it freely to point at a different hub.

## Adding a box later

One entry in the inventory, then:

```
ansible-playbook site.yml -K --limit  
box-c,localhost
```

The `localhost` again — without it, the new box provisions but never appears in your fleet list.

## Pinning a version

`hive_ref` in your vars file is what the boxes check out. It takes a branch, a tag, or a commit. Set it to something specific when you want the fleet to stop tracking your latest push, and converge to move them.

# Three Consoles

---

Everything in this chapter talks to the same hub and sees the same fleet. What changes is where you are and what you're trying to do.

The three consoles aren't tiers — there's no “basic” one and no “pro” one. They're shaped for different moments. The terminal is for when you're already in a terminal. The Mac panel is for keeping half an eye on four machines while you work on something else. The phone is for the twenty minutes you're away from a desk and an agent has gone quiet in a way that might mean it's finished or might mean it's stuck.

## The terminal

One command, `hive`, with subcommands. It reads a two-line config file naming your hub and holds no state of its own.

```
$ hive status
box-a          --          (no sessions)
box-b          --          (unreachable)
  error sending request for url (...)
*box-a          demo      running   claude
  /home/you/proj
  Running tests now -- 14 passed, 2
  failing...
```

One line per session: host, task, state, which agent, working directory. The indented line underneath is the last thing that agent printed, captured straight from its terminal. A star in the first column means you marked the session as one to watch.

Two rows there aren't sessions at all. A host with nothing running on it says (no sessions). A host the hub could not reach at all says (unreachable), followed by the actual network error. Both are deliberate — a box that vanishes from a fleet listing is a box you forget to fix.

The terminal is the right tool when you already have one open, when you want to script something, or when you want output you can pipe. It's the only console that can start the daemons themselves, and it's the one that keeps working when you're SSH'd into a machine somewhere with nothing else available.

## The Mac panel

A native application — not a browser window — that shows your whole fleet and refreshes itself as the hub learns things.

The fleet is a table or a grid of tiles, your choice, and you can narrow it by state, host, agent, or starred. With four boxes and a dozen sessions that filtering stops being a nicety: most of the time you want the three sessions that are actually running and not the nine that finished on Tuesday. Filters persist between launches, as do the theme and text size, because a panel you leave open all day should look the way you like.

Each session offers what makes sense for its state. A running session can be chatted with, attached to, or stopped. A finished one can be resumed. Any session can be restarted, killed outright, or opened as a plain SSH shell in its working directory — useful when you want to

look at what the agent did without interrupting it.

Chat is the part that matters

Click a session and you get its conversation, rendered properly: Mark-down formatted, mathematics typeset, code in blocks rather than as a wall of escape sequences. Underneath, it's the same transcript the agent is writing to disk on the box, streamed to you as it grows. There's a compose box at the bottom. What you type gets typed into the agent's terminal.

This is a better way to read an agent than attaching to its terminal is. Attaching shows you a scrolling pane you can't search and can't scroll far back in. The panel shows you the conversation.

When an agent stops to ask permission — to edit a file, run a command — the panel says so: *waiting on a permission prompt*. It won't answer for you. Approving means attaching a real terminal, because the prompt is a menu in the agent's own interface rather than a message in the conversation. *Running Agents* covers how to avoid these prompts entirely for sessions you plan to leave alone.

## Spawning

The panel's spawn dialog asks for a host, a directory, a task name, and which agent. The directory field browses the *box*'s filesystem, not your laptop's, which spares you guessing at remote paths.

## The phone

Open `http://<hub>/hive/` on your phone. That's the entire install — it's a web page the hub serves, so there's nothing to keep updated. Add it to your home screen and it behaves close enough to an app.

You get the fleet as a stack of cards, one per session, filterable by host or down to just the starred ones. Each card carries the session's state, its agent, and its directory. From a card you can star, chat, stop, restart, or get the SSH command for that session copied to your clipboard.

Chat is the reason to have it. Full-screen, the same rendered conversation the Mac panel shows, live. An agent that's been running for an hour while you were at lunch can be read and answered from a phone, and that changes which jobs feel safe to start — a long unattended run stops being a commitment to stay near your desk.

You can also start new work from the phone. Spawn is in the header and browses the box's filesystem the same way the panel does.

Two things the phone deliberately doesn't do. There's no terminal in the browser, so attach hands you the SSH command to paste into whatever terminal app you keep on your phone. And there's no kill — the destructive one is left to the two consoles where you're less likely to hit it with a thumb.

The phone console is served over plain HTTP on your tailnet, with no login. That's the same trust model as the rest of hive (*The Fleet*), but it's worth noticing on a device that leaves the house. A phone that's signed in to your tailnet is a phone that can drive your fleet.

## Attach: the escape hatch

Sometimes you need the agent's actual terminal — to answer a permission prompt, to use a feature the chat view doesn't expose, to see exactly what it's showing.

Attach is the one place hive still uses SSH. The hub hands back a command:

```
ssh box-a -t 'tmux attach -t hive-demo'
```

What happens next depends on the console. The Mac panel runs it in a new terminal window — iTerm2 by default, or Terminal, Ghostty, or Warp if you'd rather. The `hive` command runs it in place. The phone copies it for you to paste.

You end up in the agent's `tmux` session, exactly as if you'd SSH'd in yourself. Detach with `Ctrl-b d` and the agent keeps running, which is the thing to remember: detaching leaves it alone, and closing the window is also fine.

While you're attached and typing, `hive` will refuse to send messages to that session from any console — it checks for recent keyboard activity and declines rather than paste into the middle of what you're doing. If a say from the panel or phone reports that a human is attached, that human is you, in another window.

## Which one to reach for

| You want to                  | Reach for               |
|------------------------------|-------------------------|
| See everything at once       | the panel               |
| Read a long conversation     | the panel, or the phone |
| Answer a permission prompt   | attach                  |
| Check in while you're out    | the phone               |
| Script it, or pipe it        | the terminal            |
| Start or install the daemons | the terminal            |

Most people use all three within a day and stop thinking about which is which. Starting agents, and deciding what happens when they finish, comes next.

# Running Agents

---

Starting an agent takes four decisions: which box, what to call it, which directory, and which agent. Everything else about a session follows from those.

```
hive run box-a period-index  
    /home/you/lab/period-index
```

That starts Claude on `box-a`, in that directory, under the name `period-index`. The panel and phone ask the same four questions in a dialog; the directory field there browses the box's own filesystem, which saves you guessing at a remote path.

## The four decisions

**Which box.** Any box in your fleet. Nothing stops two agents from running on the same machine, and nothing balances them for you — if one box is doing three compiles and another is idle, that's your call to make.

**What to call it.** The task name is how you'll refer to this session forever after, so pick something you'll recognize in a list next week. Letters, numbers, hyphens, and underscores; no spaces. Names are unique per box, and starting a second session with a live name is refused rather than silently doing something surprising.

**Which directory.** An absolute path on the box, which must already exist. This becomes the agent's working directory, and it's recorded, so resuming later doesn't require you to remember it.

**Which agent.** Claude unless you say otherwise:

```
hive run box-a my-task /home/you/proj  
--provider codex
```

## Passing arguments to the agent

Anything after the hive arguments goes to the agent itself, untouched:

```
hive run box-a my-task /home/you/proj --  
--permission-mode acceptEdits
```

hive doesn't interpret these. Model selection, permission modes, whatever flags the agent supports — they pass straight through.

## The permission trap

This is the one that costs people an afternoon, so it gets its own section.

A coding agent normally stops and asks before editing a file or running a command. That's correct when you're sitting in front of it. It is a disaster for a detached session, because there's nobody to answer: the agent stops, and it stays stopped. You come back in three hours to an agent that has done nothing since minute four and is still politely waiting.

A detached agent blocked on a permission prompt waits forever. It does not time out, does not assume yes, and does not fail. Any session you start and walk away from needs the permission question settled *before* you start it.

Two ways to settle it. Pre-approve in the project's own agent settings, which is the durable answer for a repository you work in often. Or pass a permission mode at spawn time, which is the quick answer:

```
hive run box-a my-task /home/you/proj --  
--permission-mode acceptEdits
```

Both consoles will tell you when a session is blocked — the panel and the phone show a *waiting on a permission prompt* banner in the chat view. Answering requires attaching a terminal, since the prompt is a menu in the agent's interface rather than a message in the conversation.

## Stop, restart, kill, resume

Four verbs, and the difference between them is what happens to the record.

**Stop** ends the `tmux` session and keeps everything else. The agent is gone; the session's metadata, its working directory, and its conversation all remain. It shows as `exited` and can be resumed.

**Resume** starts the agent again, in the same directory, continuing the same conversation — it doesn't start over. This is why stop is safe: it's a pause, not a discard.

**Restart** is stop followed by resume. Use it when an agent is wedged but you want the conversation.

**Kill** stops the session and deletes the record — metadata, star, everything hive knew about it.

```
hive stop box-a my-task  
hive restart box-a my-task  
hive kill box-a my-task
```

Kill is not reversible. After it, hive cannot resume that session — the recorded directory and conversation link are gone. The agent’s own conversation file still exists on the box, but reconnecting to it by hand is work (*Chat and Transcripts*). When you mean “pause,” use stop.

## What happens on the box

Worth knowing, because it explains what survives what.

Starting a session writes a small file describing it and creates a `tmux` session running the agent. The agent runs as you, on that box, with your environment and your logins — hive starts it and then gets out of the way.

When the agent exits on its own, the `tmux` session doesn’t disappear. It drops to a shell with a note saying the agent exited and the conversation was saved. This is deliberate: it means you can attach afterwards and look at what happened.

Sessions do *not* survive a reboot. The `tmux` sessions are gone, the records remain, and everything shows as `exited`. Resuming them after the box comes back is a per-session decision — hive won’t do it for you.

## Stars

With a dozen sessions across four boxes, most of them finished, stars are how you keep the two you care about visible.

```
hive label box-a my-task star
hive label box-a my-task unstar
```

In the panel and on the phone it’s the star on the session, and both can

filter to starred only. The star lives on the box alongside the session's record, so it's the same star from every console.

## Looking around a box before you start

To see what's on a box without SSHing to it:

```
hive fs box-a --path /home/you/lab
```

This is the same directory listing the spawn dialogs browse. It's read-only and it's for finding the path you want to point an agent at.

# Chat and Transcripts

---

Reading what an agent said, and answering it, from a machine that isn't the one it's running on.

This works well, and it works by a mechanism worth understanding, because the mechanism has one failure mode that looks like a bug and isn't. The first half of this chapter is the part you'll use daily. The second half is what to do when the conversation you're reading stops being the conversation the agent is having.

## Saying something

```
hive say box-a my-task --text "run the
    tests before committing"
```

In the panel or on the phone, it's the compose box at the bottom of the chat view.

What happens on the box is exactly what it sounds like: your text is typed into the agent's terminal and submitted, as though you'd attached and typed it. There's no separate message channel — hive is driving the same interface you would drive by hand.

That has consequences worth knowing. The session has to be running; you can't queue a message for an agent that has exited. And because it's a keyboard, hive clears whatever was half-typed at the prompt

before sending, so a message doesn't get glued onto an abandoned fragment.

If you're attached in a terminal and actively typing, a say from another console is refused rather than pasted into the middle of your input. The message says a human is attached. That human is you, in another window — detach, or finish typing, and send it again.

## Reading the conversation

The panel and the phone show the conversation rendered — Markdown formatted, mathematics typeset, code in blocks — and keep it live as the agent works. From the terminal you can fetch it as data:

```
hive transcript box-a my-task --tail 50
```

Underneath, none of this is scraped from the terminal. The agent writes its conversation to a file on the box as it goes, and hive reads that file. This is why the chat view can show you a properly formatted conversation with scrollbar rather than a screenshot of a pane: it's reading the real record, not the display.

The consoles keep up by streaming. The box re-reads the file every couple of seconds and pushes anything new through the hub to whoever's watching, which is why the panel updates on its own without you refreshing. Nothing is sent when nothing has changed.

## When the conversation goes stale

Here's the part that confuses people.

A session's record includes an identifier for the conversation file it should read — a *binding*. Normally hive sets this when it starts the

agent and it stays correct for the life of the session.

But the agent can change which conversation it's in without telling anyone. The clearest case: you attach to the terminal and use the agent's own resume command to jump to a different conversation. The agent obligingly switches. The session's record still points at the old one.

Nothing errors. The chat view keeps showing you a conversation, still correctly rendered, live-looking — and it's the wrong one. Messages you send still go to the terminal and still reach the agent, so the agent responds to you, but you can't see the response. That asymmetry is the symptom.

The tell: the conversation you're reading stopped changing, but the session is clearly running and replies to what you send. If the view has simply frozen with the agent frozen too, that's a stuck agent, not a stale binding.

### Fixing it

Ask what's available:

```
$ hive bind box-a my-task
box-a my-task bound=3f2a...
      suggested=9c41... pane=9c41...
* 9c41... my-task
  2026-09-16T14:22:08Z live suggested
3f2a... my-task
  2026-09-16T11:03:44Z current not the
session in the tmux pane
```

This lists every conversation that could plausibly belong to this session. The one marked **current** is what hive is reading now. The

one marked `live` is what the agent is actually in — `hive` determines this by looking at the agent's own terminal, which is why it can be confident rather than guessing. The starred line is what it suggests, and when a live one exists, that's the suggestion.

Point the session at it:

```
hive bind box-a my-task --to 9c41...
```

In the panel and on the phone, the chat view has a **Bind** button showing the same list with the suggestion highlighted. Picking one updates the session's record on the box, so every console sees the correction.

Rebinding changes only which conversation `hive` reads. It does not touch the agent, interrupt it, or alter any conversation. If you pick wrong, pick again.

## When there's nothing to read yet

A brand-new session has no conversation file until the agent writes one, which it does when it first responds. A chat view that's empty for the first few seconds of a session's life is normal. If it's still empty a minute later and the agent shows as running, attach and look — the usual cause is an agent sitting at a permission prompt — the trap in *Running Agents*.

# Keeping It Running

---

A fleet is a thing you maintain. Machines reboot, versions drift, and something that worked in March stops working in June for a reason that turns out to be three months old.

This chapter is the maintenance you'll actually do and the failures you'll actually see.

## Pushing new code

When hive itself changes, the boxes need the new version. They fetch it themselves — so push first, then:

```
hive-deploy box-a box-b
hive-deploy --all
```

This pulls the pinned version on each box, rebuilds, and restarts the daemons if the binary changed. It's a narrower operation than a full converge: no packages, no toolchain, no linger. Day-to-day, it's the one you want.

hive-deploy deploys what's at your configured version *in the remote repository*. It does not deploy your working directory. Changes you haven't pushed don't exist as far as the boxes are concerned, and the symptom of forgetting is a deploy that reports success and changes nothing.

To preview without changing anything, add `-n`.

## After a reboot

A box that reboots comes back with its daemon running — that’s what the service install is for — but with no agents. The `tmux` sessions are gone; the records survive. Everything shows as `exited`.

Resuming is per-session and deliberate:

```
hive restart box-a my-task
```

If a box comes back and the daemon *isn’t* running, `linger` is the first thing to check (*Standing Up the Fleet*).

## The failures you’ll see

A host says (unreachable)

The hub couldn’t get an answer from that box. The error beside it is the real one — read it before guessing.

Three causes, in the order they’re worth checking. The daemon isn’t running:

```
ssh box-a 'systemctl --user status  
hive-host'
```

Or it’s running but the hub is asking the wrong address. The hub’s fleet list holds a Tailscale address per box, recorded at converge time; if a box changed addresses, converge again to regenerate it. Or the tailnet path itself is broken, which you can test from the hub:

```
ssh hub 'curl -s  
http://100.64.0.2:8788/v1/sessions '
```

### A host flaps in and out

Reachable, then not, then reachable. This is almost always linger: the daemon is being stopped when your login session ends and started when you log back in. Enable it (*Standing Up the Fleet*) and restart the daemon.

### hive status shows something impossible

Sessions that don't exist, missing fields, a box reporting oddly. Usually an old binary — a box running a version of `hive-host` that predates a change in what it reports. Deploy to it.

### An agent is running but does nothing

Attach and look. The usual answer is a permission prompt (*Running Agents*). The second most usual is that the agent finished a while ago and is sitting at a shell, which the session's last line will show.

### The chat view is stale

The conversation stopped updating but the agent replies to what you send: that's a stale binding, and *Chat and Transcripts* has the fix.

### Everything is unreachable at once

The hub, not the boxes. Check it:

```
ssh hub 'systemctl --user is-active  
hive-hub '
```

The agents are unaffected — this is a blindness, not an outage. If you need one urgently, SSH to its box and attach directly.

## Health at a glance

Ask the hub what it sees:

```
curl -s http://hub:8787/v1/fleet | jq .
```

This is exactly what every console renders, and it distinguishes a broken client from a broken fleet: if this is right and your panel is wrong, the problem is the panel.

## Housekeeping

Finished sessions stay in the list until you remove them, and there's no expiry. Once a week, kill what you're done with — the record only, the agent is long gone:

```
hive kill box-a old-task
```

Star the two or three that matter and filter to starred; a fleet list nobody prunes stops being read.

## Moving the hub

Move the machine into the hub group in your inventory, converge, and the generated fleet lists follow. Your laptop's `client.toml` does not — it's seeded once and then yours, so point it at the new hub by hand.

## What isn't automatic

Worth saying plainly, because each of these has surprised someone:

- Crashed agents are not restarted.
- Sessions are not resumed after a reboot.
- Stuck sessions raise no alert — nothing watches for an agent that has been waiting since Tuesday.

- Old session records are not cleaned up.
- Coding agents are not installed, updated, or re-authenticated for you.

hive shows you your fleet and lets you act on it. Deciding what should happen stays with you.

# Quick Reference

---

Everything, tersely. The chapters explain; this lists.

## Fleet and sessions

| Command                         | Does                            |
|---------------------------------|---------------------------------|
| hive status                     | one line per session, all hosts |
| hive run HOST TASK DIR          | start an agent                  |
| hive say HOST TASK --text "..." | send a message                  |
| hive stop HOST TASK             | end it, keep the record         |
| hive restart HOST TASK          | stop, then resume               |
| hive kill HOST TASK             | end it, delete the record       |
| hive attach HOST TASK           | open its terminal over SSH      |
| hive label HOST TASK star       | mark as one to watch            |
| hive label HOST TASK unstar     | unmark                          |
| hive transcript HOST TASK       | conversation as JSON            |
| hive bind HOST TASK             | list conversation candidates    |
| hive bind HOST TASK --to ID     | repoint at a conversation       |
| hive fs HOST                    | list a directory on a box       |
| hive panel                      | launch the Mac panel            |

## Flags

| Flag                         | On         | Meaning                     |
|------------------------------|------------|-----------------------------|
| <code>--home PATH</code>     | all        | use a different config root |
| <code>--provider NAME</code> | run        | claude (default) or codex   |
| <code>--resume</code>        | run        | restart a stopped session   |
| <code>-- ARGS</code>         | run        | pass the rest to the agent  |
| <code>--text TEXT</code>     | say        | the message (required)      |
| <code>--tail N</code>        | transcript | last N blocks               |
| <code>--after CURSOR</code>  | transcript | only what's new             |
| <code>--to ID</code>         | bind       | the conversation to bind    |
| <code>--path PATH</code>     | fs         | directory to list           |

## Session states

| State         | Means                                          |
|---------------|------------------------------------------------|
| running       | the agent's <code>tmux</code> session is alive |
| exited        | record kept, agent gone; resumable             |
| (no sessions) | the box is fine and has nothing running        |
| (unreachable) | the hub got no answer from that box            |

## Daemons

| Command                        | Does                                         |
|--------------------------------|----------------------------------------------|
| <code>hive host serve</code>   | run <code>hive-host</code> in the foreground |
| <code>hive host install</code> | install and start it as a service            |
| <code>hive hub serve</code>    | run <code>hive-hub</code> in the foreground  |
| <code>hive hub install</code>  | install and start it as a service            |

Both take `--bind ADDR`, `--port N`, and `--dev`. `install` also takes `--bin PATH`.

| Bind address                        | Accepted        |
|-------------------------------------|-----------------|
| A Tailscale address (100.64.0.0/10) | always          |
| 127.0.0.1 or localhost              | only with --dev |
| 0.0.0.0, ::, *                      | never           |

Ports

| Port | Where     | What                             |
|------|-----------|----------------------------------|
| 8788 | every box | hive-host                        |
| 8787 | the hub   | hive-hub                         |
| 80   | the hub   | phone console at /hive/          |
| 8789 | anywhere  | hive-web, local development only |

Configuration files

All under HIVE\_HOME, which is ~/.hive unless the HIVE\_HOME environment variable or --home says otherwise.

| File                 | Machine     | Holds                      |
|----------------------|-------------|----------------------------|
| client.toml          | laptop      | which hub to talk to       |
| hosts                | laptop, hub | the fleet list (generated) |
| host.toml            | each box    | bind address, port         |
| hub.toml             | hub         | bind, port, poll interval  |
| web.toml             | hub         | phone console server       |
| sessions/TASK.meta   | each box    | one session                |
| sessions/TASK.labels | each box    | its star                   |
| fleet/inventory.yml  | laptop      | your machines, for Ansible |
| fleet/group_vars/... | laptop      | shared deploy settings     |

Fields

| File        | Field     | Default                            |
|-------------|-----------|------------------------------------|
| client.toml | hub       | none; a name or full URL           |
| client.toml | terminal  | iterm2, terminal, ghostty, or warp |
| host.toml   | bind      | its own Tailscale address          |
| host.toml   | port      | 8788                               |
| hub.toml    | bind      | its own Tailscale address          |
| hub.toml    | port      | 8787                               |
| hub.toml    | poll_secs | 5                                  |
| hub.toml    | host_port | 8788                               |

The hosts file

One machine per line: a name, optionally followed by where to reach it. # starts a comment.

```
box-a http://100.64.0.2:8788
box-b http://100.64.0.3:8788
hub   http://100.64.0.1:8788
```

Generated from your inventory on every converge. Edit the inventory, not this.

A session record

| Key        | Is                                      |
|------------|-----------------------------------------|
| task       | the name you gave it                    |
| dir        | working directory on the box            |
| tmux       | its tmux session name                   |
| started    | when, in UTC                            |
| cmd        | the command line that started the agent |
| session_id | which conversation to read              |
| provider   | claude or codex                         |

## Environment variables

| Variable   | Does                                                    |
|------------|---------------------------------------------------------|
| HIVE_HOME  | config root; default <code>~/ .hive</code>              |
| CLAUDE_BIN | where <code>claude</code> is, if not on the usual paths |
| CODEX_BIN  | the same for <code>codex</code>                         |
| HIVE_PANEL | which panel binary <code>hive panel</code> launches     |

Without an override, agents are looked for under `~/ .local/bin`, then any Node version directories, then the system paths.

## HTTP API

Every client uses this; it is also what to reach for when diagnosing.

The hub, on 8787

| Endpoint                            | Returns                        |
|-------------------------------------|--------------------------------|
| GET <code>/v1/health</code>         | the collector is alive         |
| GET <code>/v1/fleet</code>          | every host and session, cached |
| GET <code>/v1/events</code>         | a stream: something changed    |
| ANY <code>/v1/hosts/NAME/...</code> | forwarded to that box          |

Anything a box can do is reachable through the hub by prefixing its path with `/v1/hosts/NAME`. A client never contacts a box directly.

## A box, on 8788

| Endpoint                         | Does                      |
|----------------------------------|---------------------------|
| GET /v1/health                   | the daemon is alive       |
| GET /v1/sessions                 | every session on this box |
| POST /v1/sessions                | start one                 |
| GET /v1/sessions/TASK/transcript | the conversation          |
| GET .../transcript/stream        | the conversation, live    |
| POST .../say                     | send a message            |
| POST .../stop                    | end it, keep the record   |
| POST .../restart                 | stop, then resume         |
| POST .../kill                    | end it, delete the record |
| POST .../label                   | star or unstar            |
| GET .../bindings                 | conversation candidates   |
| POST .../bind                    | repoint at a conversation |
| GET .../attach                   | the SSH command to attach |
| GET /v1/fs                       | list a directory          |

## Provisioning

| Command                         | Does                       |
|---------------------------------|----------------------------|
| ansible-playbook site.yml -K    | converge the whole fleet   |
| ... --limit box-a,localhost     | one box, plus client files |
| ... --check --diff              | show what would change     |
| ... --tags web --limit hive_hub | deploy the phone console   |
| hive-deploy box-a               | push code to one box       |
| hive-deploy --all               | push code to all of them   |
| hive-deploy -n box-a            | preview that               |

Run the playbook from the `ansible` directory of your clone.

## Tags

| Tag                       | Step                                                 |
|---------------------------|------------------------------------------------------|
| <code>packages</code>     | system packages                                      |
| <code>toolchain</code>    | Rust                                                 |
| <code>checkout</code>     | clone at the pinned version                          |
| <code>build</code>        | compile on the box                                   |
| <code>hive</code>         | install and start <code>hive-host</code>             |
| <code>hub</code>          | install and start <code>hive-hub</code>              |
| <code>web</code>          | the phone console                                    |
| <code>linger</code>       | let services survive logout (needs <code>-K</code> ) |
| <code>clienthosts</code>  | the laptop's fleet list                              |
| <code>clientconfig</code> | the laptop's <code>client.toml</code>                |
| <code>summary</code>      | report the deployed version                          |

## Deploy settings

In `fleet/group_vars/all/vars.yml`:

| Setting                         | Is                                               |
|---------------------------------|--------------------------------------------------|
| <code>hive_repo_url</code>      | where the boxes clone from                       |
| <code>hive_ref</code>           | branch, tag, or commit to deploy                 |
| <code>hive_checkout_dir</code>  | <code>/srv/lab/hive</code>                       |
| <code>hive_hub_host</code>      | written into the laptop's config                 |
| <code>hive_host_port</code>     | 8788                                             |
| <code>hive_hub_port</code>      | 8787                                             |
| <code>hive_web_base_path</code> | <code>hive</code> , the phone console's URL path |

## Building from source

From the repository root:

| Target                          | Builds                                               |
|---------------------------------|------------------------------------------------------|
| <code>make build</code>         | the three Rust binaries                              |
| <code>make install</code>       | them, into <code>bin/</code>                         |
| <code>make install-panel</code> | the Mac panel                                        |
| <code>make install-ops</code>   | <code>hive-deploy</code> onto your <code>PATH</code> |
| <code>make test</code>          | the test suite                                       |
| <code>make stack-smoke</code>   | a local hub and box, no tailnet                      |
| <code>make deploy-web</code>    | the phone console                                    |

Or, for the laptop command alone:

```
cargo install --path crates/hive-cli
```

## Running it locally

A whole fleet on one machine, for trying things out:

```
cargo run -p hive-host -- serve --dev
    --bind 127.0.0.1
cargo run -p hive-hub -- serve --dev
    --bind 127.0.0.1
printf 'hub = "http://127.0.0.1:8787"\n'
> ~/.hive/client.toml
hive status
```

`--dev` is what permits a loopback bind. Without it, both daemons refuse to start.

## When something is wrong

| Symptom                         | Start with                          |
|---------------------------------|-------------------------------------|
| One host ( <b>unreachable</b> ) | is its daemon running?              |
| A host flaps in and out         | linger                              |
| Every host unreachable          | the hub's daemon                    |
| Stale or impossible sessions    | deploy to that box                  |
| Agent runs but does nothing     | attach; likely a permission prompt  |
| Chat frozen, agent replies      | a stale binding; rebind             |
| Converge hangs on the password  | classic <b>sudo</b>                 |
| Session won't start             | is the agent installed on that box? |